Let's create a trivial example and then build it up into something realistic. We'll start by implementing a simple method that adds logging capabilities to instances of a class. We previously did this using a module—this time we'll do it using a class-level method. Here's the first iteration:

metaprogramming/logging_0.rb

```
class Example
  def self.add_logging
    def log(msg)
      $stderr.puts("#{Time.now.strftime("%H:%M:%S: ")} #{self} (#{msg})")
    end
  end

  add_logging
end

ex = Example.new
ex.log("hello")
```

produces:

```
00:12:26:  #<Example:0x0000000104988758> (hello)
```

Clearly, this is a silly piece of code. But bear with us—it'll get better. And we can still learn from it. First, notice that add_logging is a class method—it is defined in the class object's singleton class. That means we can call it later in the class definition without an explicit receiver, because self is set to the class object inside a class definition.

Then, notice that the add_logging method contains a nested method definition. This inner definition will get executed only when we call the add_logging method. The result is that log will be defined as an instance method of class Example.

Let's take one more step. We can define the add_logging method in one class and then use it in a subclass. This works because the singleton class hierarchy parallels the regular class hierarchy. As a result, class methods in a parent class are also available in the child class, as the following example shows:

metaprogramming/logging_1.rb

```
class Logger
  def self.add_logging
    def log(msg)
      $stderr.puts("#{Time.now.strftime("%H:%M:%S: ")} #{self} (#{msg})")
    end
  end
end

class Example < Logger
  add_logging
end

ex = Example.new
puts ex.log("hello")
```

produces:

```
00:12:26:  #<Example:0x0000000104cf8870> (hello)
```

Think back to the two examples at the start of this section. Both work in the same way as our add_logging example. attr_accessor is an instance method defined in class Module and so is available in all module and class definitions. has_many is a class method defined in the Base